

Aula 02

Processamento básico de imagens com OpenCV

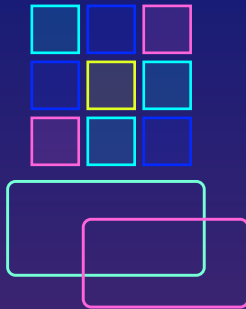
Transformando pixels em informação útil

Da matriz ao processamento

Francisco Leonel Ferreira dos Santos

Treinamento de Visão Computacional com YOLO

Caxias – MA, 2026



Por que processar uma imagem?



Melhorar visualização
contraste e legibilidade

Reduzir ruído
suavizar variações indesejadas

Destacar estruturas
bordas, formas e texturas

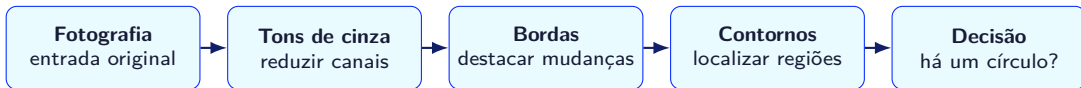
Separar regiões
objeto e fundo

Adaptar formato
tamanho, canais e representação

Preparar algoritmos
entradas consistentes

Exemplo motivador

Um documento fotografado com sombra pode precisar de correção ou limiarização antes que seu conteúdo seja analisado.



Pré-processamento

Modifica ou reorganiza a entrada para facilitar a análise.

Decisão

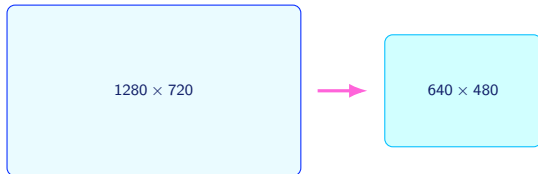
Usa características extraídas para produzir uma resposta.

Transforme a imagem com um objetivo.



```
imagem_menor = cv2.resize(  
    imagem,  
    (640, 480)  
)
```

- ▶ altera largura e altura;
- ▶ reduz ou aumenta a quantidade de pixels;
- ▶ pode perder detalhes ou introduzir interpolação.



Conexão com YOLO

Detectores normalmente trabalham com tamanhos de entrada controlados.

Mais pixels significam mais custo, não garantia de melhor resultado.

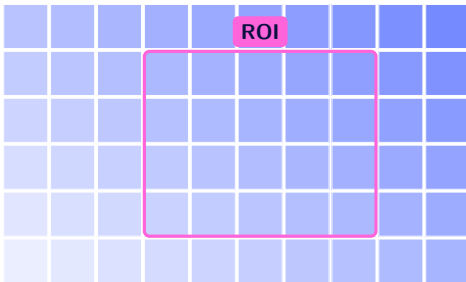


```
roi = imagem[  
    100:300,  
    200:500  
]
```

ROI

Region of Interest: área selecionada para análise específica.

- ▶ ignora regiões irrelevantes;
- ▶ reduz processamento;
- ▶ permite focar no objeto esperado.

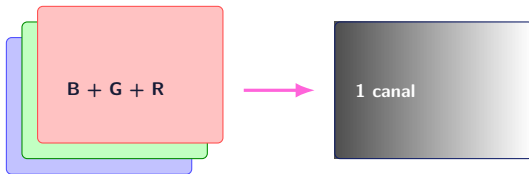


No array, primeiro selecionamos linhas (y) e depois colunas (x).



```
cinza = cv2.cvtColor(  
    imagem,  
    cv2.COLOR_BGR2GRAY  
)
```

- ▶ Reduz três canais para um.
- ▶ Mantém informação de intensidade.
- ▶ Pode ser útil quando a cor não é essencial.



Cuidado

Eliminar cor também pode remover uma pista importante da tarefa.

Exemplo numérico: de BGR para tons de cinza



Pixel lido pelo OpenCV

$$[B, G, R] = [50, 120, 200]$$

Azul = 50, verde = 120 e vermelho = 200

Combinação ponderada

$$\begin{aligned} I &= 0,114B + 0,587G + 0,299R \\ &= 0,114(50) + 0,587(120) + 0,299(200) \\ &= 5,70 + 70,44 + 59,80 \\ &= 135,94 \approx \boxed{136} \end{aligned}$$

Contribuição dos canais

B  5,70

G  70,44

R  59,80

Resultado da conversão

[50, 120, 200]
BGR



136
cinza

Não é uma média simples

O canal verde recebe maior peso porque a percepção humana é mais sensível a essa faixa de luminosidade.

O que é um filtro?



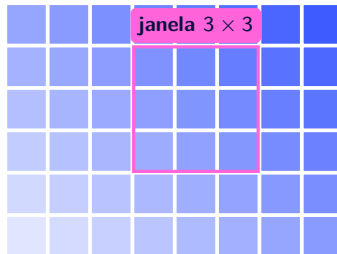
Intuição

Uma pequena matriz, chamada **kernel**, percorre a imagem e combina os valores de cada vizinhança.

Kernel de média 3×3

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Resultado: média local

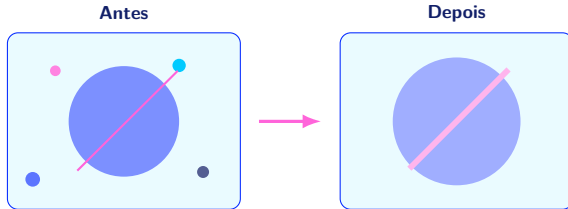


Em cada posição

O kernel gera um novo valor a partir dos pixels vizinhos.


```
suave = cv2.GaussianBlur(  
    imagem,  
    (5, 5),  
    0  
)
```

- ▶ combina informações da vizinhança;
- ▶ reduz pequenas variações e ruído;
- ▶ pode facilitar operações posteriores.



Cuidado

Suavização excessiva pode apagar bordas e detalhes importantes.

Limiarização: separando valores em grupos



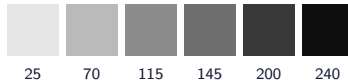
```
_, binaria = cv2.threshold(  
    cinza,  
    127,  
    255,  
    cv2.THRESH_BINARY  
)
```

Regra do exemplo

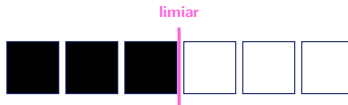
Se o pixel for maior que 127, use 255; caso contrário, use 0.

A saída possui apenas dois valores: preto ou branco.

Antes: intensidades



Depois: imagem binária



Um limiar fixo funciona em qualquer iluminação?



Cena bem iluminada



Situação favorável

Objeto e fundo apresentam separação clara de intensidades.

Mesma cena com sombra



Possível falha

Intensidades de objeto e fundo podem se sobrepor.

Alternativas

Limiarização adaptativa calcula limites locais; o método de Otsu estima automaticamente um limiar global a partir da distribuição de intensidades.

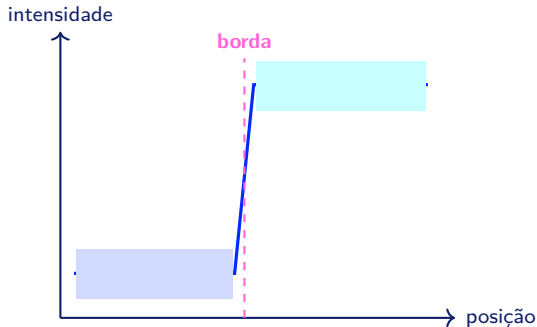


Borda

Região onde a intensidade muda de forma significativa entre pixels vizinhos.

- ▶ fornece pistas de forma e contorno;
- ▶ pode indicar limites entre objeto e fundo;
- ▶ é sensível a ruído e textura.

Exemplos: contorno de uma placa, limite de uma peça ou separação entre regiões.



Gradiente

Mede a direção e a intensidade dessa mudança.

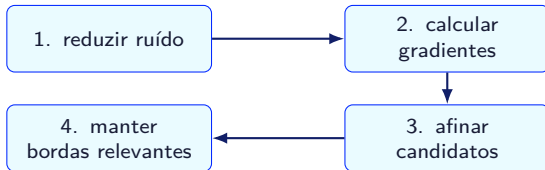


```
bordas = cv2.Canny(  
    cinza,  
    100,  
    200  
)
```

Experimento

Altere os dois limiares e observe quais bordas desaparecem ou surgem.

Pipeline em alto nível

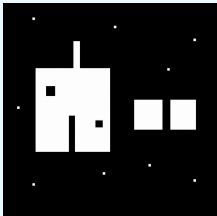


Resultado

Uma imagem binária destacando bordas selecionadas pelo algoritmo.

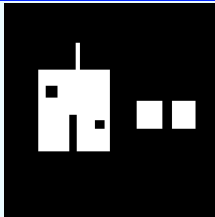


Imagem original



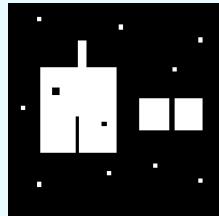
Objetos brancos, pequenos ruídos e falhas internas sobre fundo preto.

Erosão



Reduz o primeiro plano, remove pontos claros pequenos e amplia falhas.

Dilatação

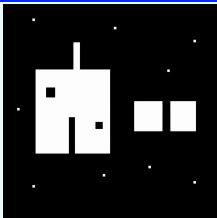


Expande o primeiro plano, aproxima componentes e reduz pequenas falhas.

A erosão contrai formas; a dilatação expande formas.

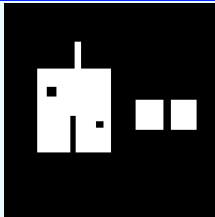


Imagem original



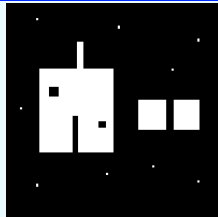
Referência para observar ruídos isolados, cavidades e separações.

Abertura



Erosão seguida de dilatação: remove ruídos claros e saliências finas.

Fechamento

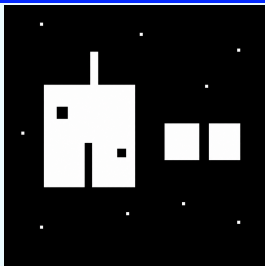


Dilatação seguida de erosão: fecha lacunas e pequenas cavidades escuras.

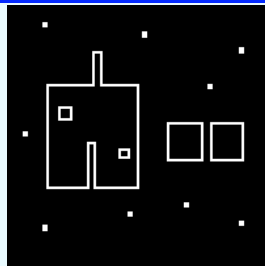
Abertura limpa o primeiro plano; fechamento preenche descontinuidades.



Imagem original



Gradiente morfológico



Definição

$$G = \text{dilatação}(I) - \text{erosão}(I)$$

Interpretação

A diferença preserva principalmente as fronteiras. A espessura depende da forma e do tamanho do kernel.



Retangular 3×3

$$K_{\text{ret}} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Atua também nas diagonais e produz alterações uniformes ao redor do objeto.

Cruz 3×3

$$K_{\text{cruz}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Prioriza vizinhos horizontais e verticais, com menor influência diagonal.

Elíptico 5×5

$$K_{\text{elipse}} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Adequado para formas curvas e transições menos quadradas.

Experimento proposto

Aplique cada kernel sobre a mesma imagem binária. Compare cantos, diagonais, pequenos ruídos e o efeito de trocar 3×3 por 5×5 .

A forma e o tamanho do kernel fazem parte da configuração do método.



1. Criar os kernels

```
import cv2
img = cv2.imread(
    "binaria.png",
    cv2.IMREAD_GRAYSCALE
)
ret = cv2.getStructuringElement(
    cv2.MORPH_RECT, (3, 3))
cruz = cv2.getStructuringElement(
    cv2.MORPH_CROSS, (3, 3))
ellipse = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (5, 5))
```

2. Aplicar as operações

```
erosao = cv2.erode(
    img, ret, iterations=1)
dilatacao = cv2.dilate(
    img, ret, iterations=1)
abertura = cv2.morphologyEx(
    img, cv2.MORPH_OPEN, cruz)
fechamento = cv2.morphologyEx(
    img, cv2.MORPH_CLOSE, ellipse)

gradiente = cv2.morphologyEx(
    img, cv2.MORPH_GRADIENT, ret)
```

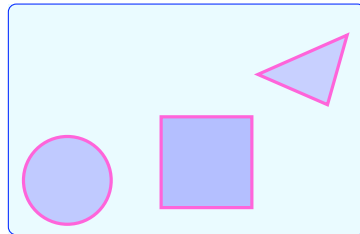
Experimento

Troque os kernels e teste uma ou duas iterações. Compare ruído, união de componentes e preservação da forma.



```
contornos, _ = cv2.findContours(  
    binaria,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

- ▶ representa fronteiras de regiões;
- ▶ permite calcular área e perímetro;
- ▶ pode gerar caixas delimitadoras.

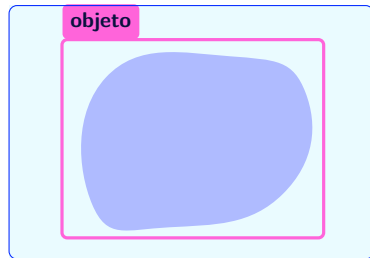


contornos externos



```
x, y, w, h = cv2.boundingRect(contorno)

cv2.rectangle(
    imagem,
    (x, y),
    (x + w, y + h),
    (0, 255, 0),
    2
)
```



Primeira localização

Estamos começando a localizar objetos, mas ainda dependemos de regras manuais.



Regra proposta: objeto vermelho + forma circular $\Rightarrow$ bola

Iluminação
a cor aparente muda

Oclusão
parte do objeto desaparece

Objetos parecidos
placas e frutas circulares

Escala
o tamanho varia

Fundo complexo
bordas se misturam

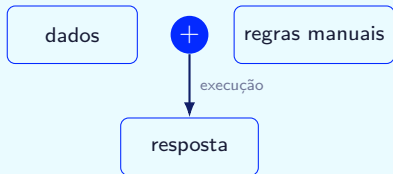
Variação real
milhares de aparências

Pergunta central

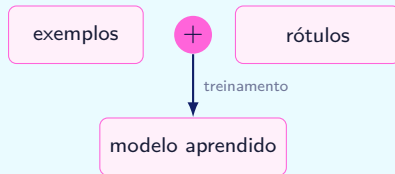
Como escrever manualmente todas as regras necessárias para reconhecer um objeto no mundo real?



Programação baseada em regras



Aprendizado de Máquina



Em vez de programar todas as regras, mostramos exemplos para que o modelo aprenda parâmetros e padrões visuais.

Próxima aula: como uma máquina aprende padrões?



```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("objetos.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 100, 200)
```

Etapas da atividade

- 1 Ler a imagem.
- 2 Redimensionar, se necessário.
- 3 Converter para cinza.
- 4 Aplicar Gaussian Blur.
- 5 Detectar bordas com Canny.
- 6 Exibir e comparar cada etapa.

Registro do grupo

Anote os parâmetros utilizados e explique o efeito observado em cada transformação.



Escolha uma imagem e tente isolar ou destacar seu objeto principal
usando apenas técnicas clássicas de processamento.

Você pode combinar

- ▶ conversão de cor;
- ▶ suavização;
- ▶ limiarização;
- ▶ bordas e morfologia.

Parte mais importante

Registre quando o método falha e proponha uma hipótese para a falha.

Não existe obrigação de obter um resultado perfeito.



- 1 O que um kernel faz ao percorrer uma imagem?
- 2 Para que serve a suavização?
- 3 O que acontece durante uma limiarização?
- 4 O que caracteriza uma borda?
- 5 Por que regras manuais podem falhar em ambientes reais?



“Pré-processar não significa melhorar sempre: significa transformar a imagem de acordo com o objetivo da tarefa.”

Tarefa opcional

Crie duas versões do mesmo pipeline com parâmetros diferentes e compare:

- ▶ quantidade e qualidade das bordas;
- ▶ preservação de detalhes;
- ▶ situações em que cada configuração funciona melhor.

Próxima aula: do processamento clássico ao Aprendizado de Máquina.